

Notas de IMP

Diego Acuña

Mayo 2026

1. Introducción

IMP es un lenguaje imperativo minimalista diseñado para estudiar formalmente la noción de ejecución, estado y semántica operacional.

A diferencia del cálculo lambda, donde la computación se expresa mediante reducción de expresiones, en IMP la computación se entiende como transformación de memoria.

El objetivo principal de IMP no es servir como lenguaje de programación práctico, sino como modelo matemático de:

- estado mutable,
- secuenciación,
- variables locales,
- control de flujo,
- semántica operacional.

IMP ocupa en semántica operacional un rol similar al que CHI ocupa en programación funcional.

1.1. Paradigma imperativo

En programación funcional, una computación se modela como evaluación de expresiones.

En programación imperativa, la computación se modela como evolución de un estado.

Por ello, el objeto semántico central de IMP es la memoria.

1.2. Computación como transformación de estado

La ejecución de un programa se expresa mediante juicios de la forma:

$$M \triangleright p \triangleright M'$$

que se leen:

“La ejecución del programa p transforma la memoria M en la memoria M' .”

Esta visión contrasta con el cálculo lambda, donde las relaciones fundamentales tienen la forma:

$$M \rightarrow_{\beta} N$$

y describen reducción sintáctica.

2. Sintaxis abstracta

2.1. Programas

Definición 2.1 (Sintaxis de programas). Los programas de IMP se definen inductivamente por:

$$p ::= x := e \mid \text{local } \bar{x} \ p \mid p_1; p_2 \mid \text{case } x \text{ of } \bar{b} \mid \text{while } x \text{ is } \bar{b}$$

Cada construcción posee una interpretación operacional:

Construcción	Significado
$x := e$	Asignación
$\text{local } x \ p$	Variables locales
$p_1; p_2$	Secuenciación
$\text{case } x \text{ of } b$	Selección por patrones
$\text{while } x \text{ is } b$	Iteración

2.2. Expresiones

Definición 2.2 (Expresiones). Las expresiones están dadas por:

$$e ::= c \ \bar{e} \mid x$$

donde c representa constructores o constantes.

2.3. Ramas

Definición 2.3 (Ramas). Las ramas utilizadas en construcciones **case** y **while** tienen la forma:

$$b ::= c \ \bar{x} \rightarrow p$$

Las variables del patrón quedan ligadas dentro del programa asociado.

2.4. Convenciones sintácticas

- La secuencia asocia a izquierda:

$$p_1; p_2; p_3 \equiv (p_1; p_2); p_3$$

- El alcance de **local** se extiende lo máximo posible.

3. Valores y memoria

3.1. Valores

Definición 3.1 (Valores). Los valores de IMP se definen inductivamente por:

$$v ::= c \ \bar{v} \mid \text{null}$$

El valor **null** representa ausencia de valor.

3.2. Memoria

Definición 3.2 (Memoria). Una memoria es una lista ordenada de asociaciones variable–valor.

La memoria se comporta como un entorno mutable.

A diferencia de los entornos funcionales tradicionales, en IMP una variable puede aparecer múltiples veces en memoria.

La búsqueda utiliza siempre la primera ocurrencia.

Ejemplo 3.3. Considérese:

$$M = [(x, 1), (y, 2), (x, 5)]$$

Entonces:

$$M_x = 1$$

La segunda aparición de x queda ocultada.

3.3. Operaciones sobre memoria

3.3.1. Búsqueda

La operación:

$$M_x$$

busca la primera ocurrencia de x .

3.3.2. Actualización

La operación:

$$M < +(\bar{x}, v)$$

actualiza la primera ocurrencia de x .

Si x no existe, la agrega.

3.3.3. Alta

La operación:

$$x + +M$$

agrega nuevas variables al comienzo de la memoria asociándoles el valor `null`.

3.3.4. Baja

La operación:

$$M - x$$

elimina la primera ocurrencia de x .

3.4. Interpretación operacional

Las memorias representan estados computacionales.

Por tanto, dos programas son equivalentes si producen exactamente las mismas transformaciones de memoria.

4. Variables locales y scope

4.1. Shadowing

Las variables locales introducen nuevas asociaciones al comienzo de la memoria.
Esto produce shadowing.

Ejemplo 4.1. Sea:

$$M = [(x, 1)]$$

Entonces:

$$x + +M = [(x, null), (x, 1)]$$

La nueva variable local oculta temporalmente a la anterior.

4.2. Restauración del entorno

Una vez finalizada la ejecución de un bloque local, la variable introducida es removida.
Esto restaura automáticamente el entorno externo.

Observación 4.2. La semántica de variables locales en IMP se asemeja a la disciplina de pila utilizada en lenguajes imperativos reales.

5. Pattern matching

5.1. Matching estructural

Los constructores determinan la forma de los valores.
Una rama:

$$\text{Cons } h \ t \rightarrow p$$

solo puede ejecutarse si el valor posee constructor **Cons**.

5.2. Variables ligadas

Las variables del patrón quedan disponibles únicamente dentro del cuerpo asociado.

6. Semántica big-step y small-step

6.1. Big-step semantics

La semántica presentada hasta ahora es una semántica big-step.
Relaciona directamente:

$$(M, p)$$

con:

$$M'$$

sin describir estados intermedios.

6.2. Small-step semantics

Alternativamente puede definirse una semántica small-step:

$$(M, p) \rightarrow (M', p')$$

que describe la ejecución como sucesión de pequeños pasos.

6.3. Comparación

Modelo	Ventaja	Desventaja
Big-step	Simplicidad	No describe intermedios
Small-step	Precisión operacional	Mayor complejidad

7. Determinismo

Teorema 7.1 (Determinismo). *La semántica operacional de IMP es determinista. Es decir, si:*

$$M \triangleright p \triangleright M_1$$

y:

$$M \triangleright p \triangleright M_2$$

entonces:

$$M_1 = M_2$$

Demostración. La demostración es por inducción estructural sobre la derivación semántica.

Cada construcción posee exactamente una regla aplicable y las evaluaciones de expresiones son deterministas.

Por hipótesis inductiva, las subejecuciones producen resultados únicos. $\square$

7.1. Importancia

El determinismo implica que los programas poseen comportamiento observacional único. Esto contrasta con semánticas concurrentes o no deterministas.

8. Propiedades metateóricas

8.1. Preservación de variables externas

Las variables locales no afectan permanentemente al entorno externo.

Proposición 8.1. *Sea:*

$$M \triangleright local\ x\ p \triangleright M'$$

Entonces toda variable distinta de x preserva su visibilidad externa.

8.2. Alcance léxico

IMP utiliza alcance léxico.

Las variables se resuelven utilizando estructura sintáctica y no contexto dinámico.

8.3. Transparencia referencial

IMP no satisface transparencia referencial.

Ejemplo 8.2. La expresión x puede evaluar a distintos valores según el momento de ejecución.

Esto contrasta radicalmente con el cálculo lambda puro.

9. Ejercicios

1. Evaluar:

$$x := S[O[]]$$

sobre una memoria vacía.

2. Ejecutar completamente:

$$local\ x\ \{x := O[]\}$$

3. Construir la derivación completa de:

$$p_1; p_2$$

4. Mostrar que la secuencia es asociativa.
5. Probar determinismo.
6. Construir un programa divergente.
7. Explicar por qué IMP no posee transparencia referencial.
8. Comparar variables ligadas en cálculo lambda con variables locales en IMP.
9. Explicar cómo modelar estado mutable funcionalmente.

10. Referencias

Referencias

- [1] Glynn Winskel, *The Formal Semantics of Programming Languages*, MIT Press.
- [2] Hanne Riis Nielson y Flemming Nielson, *Semantics with Applications*, Springer.
- [3] Benjamin Pierce, *Types and Programming Languages*, MIT Press.
- [4] Gordon Plotkin, *A Structural Approach to Operational Semantics*.
- [5] John Reynolds, *The Discoveries of Continuations*.